

C S D 6 2 2

# Pattern Recognition & Data Mining

*Master of Science Programme · Graduate Course*

---

**Topics:** Feature Extraction · Classification · Clustering · Neural Networks · SVMs · Association Rules · Evaluation

Semester Overview · 35 Lecture Slides

**Tutor:** Dr A.J Fofanah



# Course Overview



## Module 1: Foundations

Pattern recognition pipeline, Bayesian decision theory, feature spaces, types of learning.



## Module 2: Classification

k-NN, Naive Bayes, Decision Trees, SVM (kernel trick), ensemble methods.



## Module 3: Neural Networks

Perceptrons, backpropagation, CNNs, RNNs, LSTMs, deep learning architectures.



## Module 4: Unsupervised Learning

k-Means, hierarchical clustering, DBSCAN, PCA, t-SNE, UMAP.



## Module 5: Association Rules

Support, confidence, lift, Apriori algorithm, FP-Growth, rule evaluation.



## Module 6–7: Engineering & Eval

Feature engineering, selection, imbalanced data, model evaluation, cross-validation.

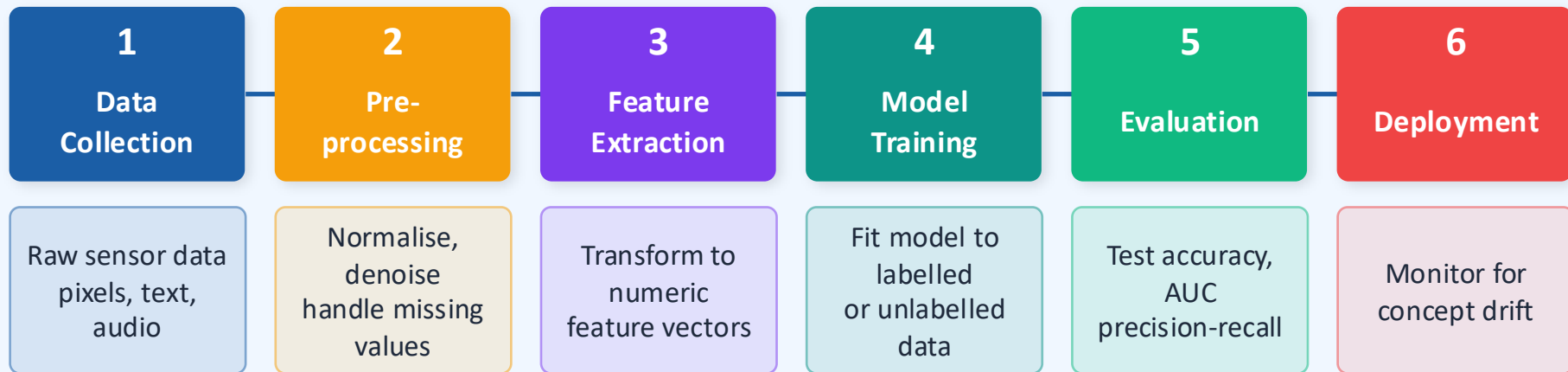


MODULE 1

# Foundations of Pattern Recognition

*Bayes' theorem, feature spaces, and the classification framework*

# The Pattern Recognition Pipeline



**Example:** Medical X-ray classifier → collect 200K labelled X-rays → resize & normalise pixels → extract CNN features → fine-tune ResNet-50 → evaluate AUC on held-out test set → deploy as clinical API.

# Bayesian Decision Theory

$$P(c \mid x) = P(x \mid c) \cdot P(c) / P(x)$$

$P(c \mid x)$

**Posterior**

Probability of class  $c$   
given observed feature  $x$

$P(x \mid c)$

**Likelihood**

Probability of  $x$  given  
that the class is  $c$

$P(c)$

**Prior**

Prior belief about  
how common class  $c$  is

$P(x)$

**Evidence**

Normalisation constant  
(marginal likelihood)

**Bayes Optimal Rule:**  $\hat{c} = \operatorname{argmax}_c P(x \mid c) \cdot P(c)$  — no other classifier achieves lower error when class-conditional densities are known. The Bayes error rate is the absolute lower bound.

## Feature Spaces & The Curse of Dimensionality

### Continuous

Height: 1.75m  
Temperature: 37°C

Encoding: Normalise

### Ordinal

Education:  
Primary < Degree

Encoding: Integer  
(1,2,3)

### Nominal

Blood type:  
A, B, AB, O

Encoding: One-hot  
encode

### Text

Customer  
reviews

Encoding: TF-IDF /  
BERT

### ⚠ The Curse of Dimensionality

- Sparsity: data becomes exponentially sparse as dimensions grow — 'nearest' neighbours may be far away
- Distance concentration: all pairwise distances converge, breaking distance-based methods (k-NN, k-Means)
- Overfitting: more features → model memorises training noise → poor generalisation

**Rule of thumb: Required samples  $\propto 2^n$  (exponential in n dimensions)**



MODULE 2

# Classical Classification Algorithms

*k-NN · Naive Bayes · Decision Trees · SVM · Ensembles*

# k-Nearest Neighbours (k-NN)

- 1 Given query  $x_q$  and training set  $\{(x_1, y_1), \dots, (x_m, y_m)\}$
- 2 Compute distance  $d(x_q, x_i)$  to every training point
- 3 Select  $k$  training points with smallest distance
- 4 Assign class with most votes among  $k$  neighbours

**Choosing  $k$ :**  $k=1 \rightarrow$  overfit, large  $k \rightarrow$  underfit. Start with  $k = \sqrt{m}$ , tune via CV. Use odd  $k$  for binary problems to break ties.

## Euclidean

$$\sqrt{\sum (x_i - y_i)^2}$$

Continuous, equal scale

## Manhattan

$$\sum |x_i - y_i|$$

Sparse/high-dim data

## Cosine

$$1 - x \cdot y / (||x|| \cdot ||y||)$$

Text similarity

## Hamming

$$(1/n) \sum [x_i \neq y_i]$$

Binary/categorical

MNIST with  $k=3$ : ~97% accuracy on 70,000 images — but  $O(m \cdot n)$  prediction time limits scalability.

# Decision Trees — Splitting Criteria

## Entropy

$$H(S) = -\sum p_i \log_2 p_i$$

Used by: ID3, C4.5  
Measures disorder; 0=pure node

$$\text{Information Gain} = H(S) - \sum_v (|S_v|/|S|) \cdot H(S_v)$$

## Gini Impurity

$$\text{Gini}(S) = 1 - \sum p_i^2$$

Used by: CART, sklearn  
Prob. of misclassifying random item

$$\text{Gini Gain} = \text{Gini}(\text{parent}) - \sum \text{weighted Gini}(\text{child})$$

## Variance Reduction

$$\text{Var}(S) - \sum w_v \text{Var}(S_v)$$

Used by: CART (regression)  
Measures target variance drop

Best split = maximum variance reduction

*Pruning: Pre-pruning (max\_depth, min\_samples\_split) or Post-pruning (reduced error) prevents overfitting.*

# Support Vector Machines (SVM)

Maximise  $2/||w||$  subject to  $y_i(w \cdot x_i + b) \geq 1 \quad \forall i$

## Linear

$$K(x, z) = x \cdot z$$

Text, high-dim sparse

## Polynomial

$$K(x, z) = (\gamma x \cdot z + r)^d$$

Image features,  
interactions

## RBF (Gaussian)

$$K(x, z) = \exp(-\gamma ||x - z||^2)$$

General-purpose  
default

## Sigmoid

$$K(x, z) = \tanh(\gamma x \cdot z + r)$$

Neural network-like

## Hard Margin SVM

Linearly separable data  
Find exact maximal margin  
No misclassification allowed

## Soft Margin SVM

Non-separable data  
Allow slack variables  $\xi_i \geq 0$   
C controls bias-variance trade-off

## Kernel Trick

Map to high-dim space  $\phi(x)$   
Using  $K(x, z) = \phi(x) \cdot \phi(z)$   
No explicit mapping needed!

# Ensemble Methods

## Bagging

Bootstrap Aggregating

- Train B models on bootstrap samples
- Random Forest: add random feature subsets
- Combines by majority vote / averaging
- ↓ Variance without increasing bias

Examples: Random Forest, Extra Trees

## Boosting

Sequential Correction

- Train models sequentially
- Each model targets previous errors
- AdaBoost: re-weight samples
- XGBoost: fit gradient residuals

Examples: AdaBoost, XGBoost, LightGBM

## Stacking

Meta-Learning

- Train diverse base models
- Use out-of-fold predictions
- Meta-learner combines outputs
- Typically 3–5 diverse base models

Examples: RF + SVM + k-NN → LogReg

*Industry reality: XGBoost and LightGBM consistently win tabular ML competitions. Random Forest is the go-to baseline.*



MODULE 3

# Neural Networks & Deep Learning

*Backpropagation · CNNs · RNNs · LSTMs · Transfer Learning*

# The Artificial Neuron & Activation Functions

$$z = w \cdot x + b \rightarrow a = f(z)$$

## Sigmoid

$$1/(1+e^{-z})$$

**Range: (0,1)**

Binary classification output  
Vanishing gradient —  
avoid in hidden layers

## ReLU

$$\max(0, z)$$

**Range: [0,∞)**

Default hidden layer activation  
Fast, avoids vanishing gradient

## Softmax

$$e^{z_i} / \sum e^{z_j}$$

**Range: (0,1)**

Multi-class output  
Outputs sum to 1 (probabilities)

## GELU

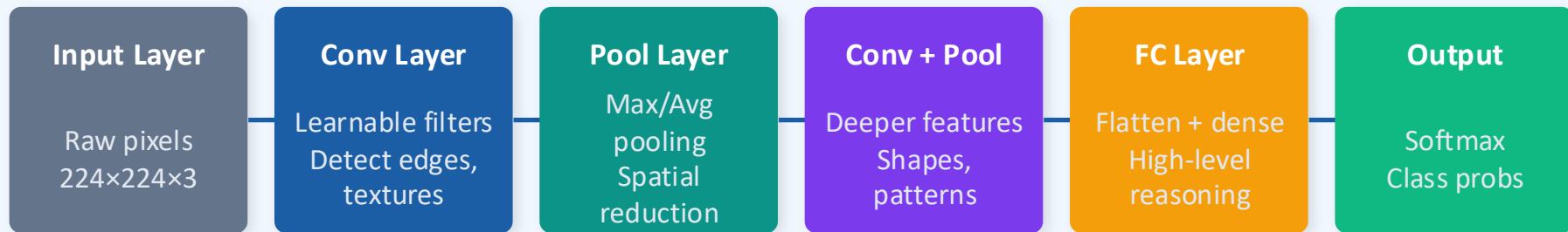
$$z \cdot \Phi(z)$$

**Range: (-∞,∞)**

Transformer models (BERT, GPT)  
Smooth ReLU approximation

**Backpropagation: Forward pass (compute loss) → Backward pass (chain rule:  $\partial L / \partial w$ ) → Update:  $w \leftarrow w - \eta \cdot \partial L / \partial w$**

# Convolutional Neural Networks (CNNs)



## AlexNet (2012)

Started deep learning era; ReLU + Dropout + GPU

**Top-5 error: 15.3%**

## VGGNet (2014)

Uniform 3×3 convolutions, simple and deep

**Top-5 error: 7.3%**

## ResNet (2015)

Skip connections — enables 100+ layers

**Top-5 error: 3.57%**

## EfficientNet (2019)

Compound scaling: depth + width + resolution

**Top-5 error: 1.8%**

**Transfer Learning:** Take ImageNet-pretrained ResNet-50, replace the final layer, fine-tune on your dataset — achieves 90%+ accuracy with only 1,000 labelled images.



MODULE 4

# Unsupervised Learning Clustering & Dimensionality Reduction

*k-Means · DBSCAN · Hierarchical · PCA · t-SNE · UMAP*

# k-Means Clustering

## 1. Initialise

Place  $k$  centroids (k-Means++ for better init)

## 2. Assign

$c_i = \operatorname{argmin}_k ||x_i - \mu_k||^2$  for each point

## 3. Update

$\mu_k = \text{mean of all points assigned to cluster } k$

## 4. Repeat

Iterate steps 2–3 until assignments stabilise

**Objective: Minimise WCSS**  $= \sum_k \sum_{i \in C_k} ||x_i - \mu_k||^2$  (Within-Cluster Sum of Squares)

### Elbow Method

Plot WCSS vs  $k$  — choose  $k$  at the 'elbow' where improvement rate sharply decreases.

### Silhouette Score

$s(i) = (b(i) - a(i)) / \max(a, b)$ . Ranges  $-1$  to  $+1$ . Choose  $k$  maximising average silhouette.

### k-Means++ Init

Sample centroids proportional to distance<sup>2</sup> from existing centroids — better convergence, lower WCSS.

*Limitation: k-Means assumes spherical clusters of similar size. Fails on non-convex shapes or varying densities → use DBSCAN.*

# DBSCAN & Hierarchical Clustering

## DBSCAN

**$\epsilon$  (epsilon):** Neighbourhood radius for density

**MinPts:** Min. points to be a core point

**Core Point:**  $\geq$  MinPts neighbours within  $\epsilon$

**Border Point:** Non-core, within  $\epsilon$  of core point

**Noise Point:** Neither core nor border — outlier

- ✓ Arbitrary shape clusters
- ✓ No  $k$  required
- ✓ Detects outliers automatically

## Hierarchical (Agglomerative)

**Single Linkage:**  $\min(d(a,b))$  — tends to chain

**Complete Linkage:**  $\max(d(a,b))$  — compact clusters

**Average (UPGMA):**  $\text{mean}(d(a,b))$  — good general choice

**Ward:** min WCSS increase — most popular

Output: Dendrogram — cut at any height for any  $k$   
No retraining needed to try different  $k$  values!

## Dimensionality Reduction: PCA · t-SNE · UMAP

### PCA

Linear

1. Centre data (subtract mean)
2. Compute covariance matrix  $\Sigma$
3. Eigendecomposition:  $\Sigma = V\Lambda V^T$
4. Project onto top-k eigenvectors

Eigenfaces: 4,096-dim face images  $\rightarrow$  100 PCs = 40x reduction

### t-SNE

Non-linear (Visualisation)

1. Model high-dim similarity with Gaussian
2. Model low-dim similarity with t-distribution
3. Minimise KL divergence via gradient descent
4. Output 2D/3D — clusters become visible

Cannot project new points; axes have no meaning — visualisation only!

### UMAP

Non-linear (Modern)

1. Build fuzzy topological graph of high-dim data
2. Optimise low-dim embedding to match
3. Preserves local AND global structure
4. Scales to millions; usable as preprocessing

Faster than t-SNE; better global structure; can project new points



MODULE 5

# Association Rule Learning

*Support · Confidence · Lift · Apriori · FP-Growth*

# Support, Confidence, and Lift

## Support

$$|T(XUY)| / |T|$$

How often items appear together in all transactions

**Range: [0, 1]**

High support = common co-occurrence

## Confidence

$$\text{Support}(XUY) / \text{Support}(X)$$

Precision of the rule  $X \rightarrow Y$  (if X, how often also Y?)

**Range: [0, 1]**

High confidence = reliable rule

## Lift

$$\text{Confidence}(X \rightarrow Y) / \text{Support}(Y)$$

How much more likely Y given X versus by chance alone

**Range: [0, ∞)**

Lift > 1: positive correlation  
Lift = 1: independent  
Lift < 1: negative correlation

Example: In 1,000 transactions, bread→butter: Support=0.2, Confidence=0.67, Lift=1.67 → 67% more likely to buy butter if bread is in basket.

# Apriori Algorithm vs FP-Growth

**Antimonotone Property: if  $\text{Support}(X) < \text{minsup} \rightarrow \text{Support}(X \cup Y) < \text{minsup}$  for any  $Y$  (PRUNE!)**

## Apriori Algorithm

1. Find frequent 1-itemsets ( $L_1$ ) — scan DB
2. Join  $L_{k-1}$  pairs to generate candidates  $C_k$
3. Prune  $C_k$  using antimonotone property
4. Scan DB to count  $C_k$  support  $\rightarrow L_k$
5. Repeat until no new frequent sets

**⚠ Multiple DB scans · huge candidate sets**

## FP-Growth Algorithm

1. Scan DB  $\rightarrow$  find frequent 1-items, sort by freq
2. Scan DB  $\rightarrow$  build compressed FP-tree
3. Mine conditional FP-trees recursively
4. No candidate generation needed!
5. Only 2 database scans total

**✓ 10–100× faster than Apriori on dense data**

**FP-Growth is always preferred over Apriori for large, dense datasets — same output, dramatically less computation.**



MODULES 6 & 7

# Feature Engineering & Model Evaluation

*Data preparation · Selection · Metrics · Validation · Bias-Variance*

# Feature Engineering & Selection

## Missing Values

- Listwise deletion (MCAR only)
- Mean/Median/Mode imputation
- MICE (Multiple Imputation)
- Add 'is\_missing' indicator flag

## Categorical Encoding

- One-hot ( $k-1$  columns, no order)
- Ordinal (preserves order)
- Target encoding (mean of target)
- Embeddings (high-cardinality)

## Feature Scaling

- Min-Max  $\rightarrow [0,1]$
- Z-score  $\rightarrow \text{mean}=0, \text{std}=1$
- Robust Scaler (IQR-based)
- Log transform (right-skewed)

## Feature Selection

- Filter: MI,  $\text{Chi}^2$ , ANOVA F-test
- Wrapper: RFE, forward/backward
- Embedded: Lasso (L1), RF importance
- Goal: remove noise, reduce overfit

# Classification Metrics

Confusion Matrix

|          | Pred + | Pred - |
|----------|--------|--------|
| Actual + | TP     | FN     |
| Actual - | FP     | TN     |

Accuracy

$$(TP+TN)/(TP+FP+FN+TN)$$

Precision

$$TP/(TP+FP)$$

Recall

$$TP/(TP+FN)$$

F1 Score

$$2 \times (P \times R) / (P + R)$$

**⚠ Imbalanced classes:** A classifier predicting 'negative' for all samples achieves 99.9% accuracy on a 0.1% fraud dataset — yet catches zero frauds! Use Recall, F1, MCC, or AUC-PR instead.

**Bias-Variance:** Expected Error = Bias<sup>2</sup> + Variance + Noise. Underfitting = high bias (too simple). Overfitting = high variance (memorises noise). Regularisation, more data, or simpler models reduce variance; more complex models reduce bias.

## Cross-Validation & Hyperparameter Tuning



**CV Score = Mean  $\pm$  Std of k fold scores (typically k = 5 or 10)**

### Grid Search

Exhaustive search over all combinations  
Guaranteed best in grid — scales poorly

### Random Search

Random sampling from distributions  
More efficient when few params matter

### Bayesian (Optuna)

Probabilistic surrogate model  
5–10× fewer evaluations than grid

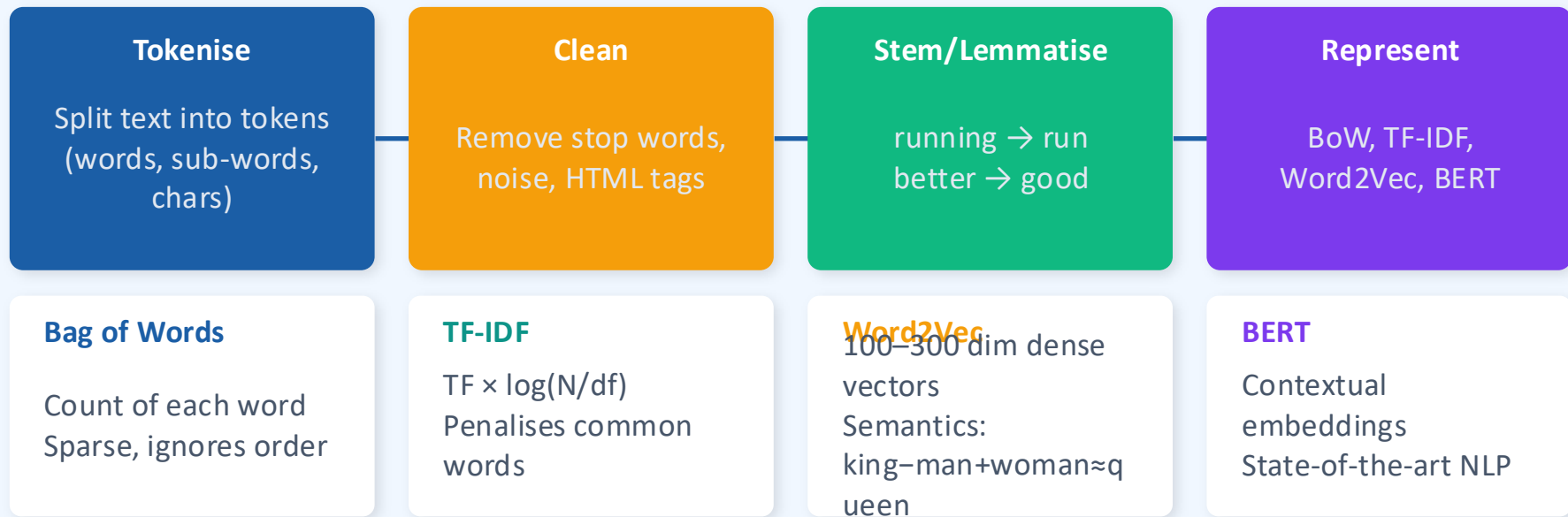


MODULES 8 & 9

# Text Mining, Web Mining & Ethics in Data Mining

*NLP · PageRank · Stream Mining · Fairness · Privacy · Explainability*

# Text Mining & NLP Pipeline



**Applications:** Sentiment analysis · Spam filtering · Topic modelling (LDA) · Named entity recognition · Question answering · Machine translation

# Responsible Data Mining

## Historical Bias

Training data reflects past discrimination — models perpetuate it

## Representation Bias

Under-represented groups → poor performance on those groups

## Feedback Loop

Model predictions influence future training data (self-reinforcing)

## Measurement Bias

Proxy variables measured differently across demographic groups

## Demographic Parity

Equal positive prediction rates across groups

## LIME

Local interpretable surrogate — explains any black-box prediction

## SHAP Values

Shapley values — marginal contribution of each feature

## Differential Privacy

Calibrated noise — protects individual data in outputs

GDPR grants a 'right to explanation' for automated decisions. SHAP and LIME are the industry standard tools for complying with this requirement.

# Real-World Applications

## Healthcare

### Tech: CNN + Transfer Learning

Detect tumours in radiology images with radiologist-level accuracy. Classifying diabetic retinopathy from retinal scans.

## Finance

### Tech: XGBoost + SHAP

Credit card fraud detection: classify 1M transactions/min. SHAP explains each decision for regulatory compliance.

## E-Commerce

### Tech: FP-Growth + Collab. Filtering

Market basket analysis: 'Customers who bought X also bought Y'. Netflix-style recommendation engines.

## NLP / Search

### Tech: BERT + k-Means

Document clustering, semantic search, sentiment monitoring. Real-time social media trend detection.

## Autonomous Systems

### Tech: ResNet + YOLO

Object detection for self-driving cars: identify pedestrians, vehicles, and road signs in real time at 30fps.

## Cybersecurity

### Tech: DBSCAN + Isolation Forest

Network anomaly detection: identify unusual traffic patterns as potential intrusions without prior labels.

# Algorithm Comparison Reference

| Algorithm     | Type         | Training                            | Prediction                | Interpretable? | Best For                             |
|---------------|--------------|-------------------------------------|---------------------------|----------------|--------------------------------------|
| k-NN          | Supervised   | $O(1)$ — lazy                       | $O(m \cdot n)$            | Yes            | Small datasets, mixed types          |
| Naive Bayes   | Supervised   | $O(m \cdot n)$                      | $O(n \cdot k)$            | Yes            | Text classification, fast baseline   |
| Decision Tree | Supervised   | $O(m \cdot n \cdot \log m)$         | $O(\text{depth})$         | Yes            | Mixed features, requires explanation |
| SVM + RBF     | Supervised   | $O(m^2 - m^3)$                      | $O(\text{SVs} \cdot n)$   | No             | Small/medium high-dim datasets       |
| Random Forest | Supervised   | $O(B \cdot m \cdot n \cdot \log m)$ | $O(B \cdot \text{depth})$ | Partial        | Tabular data, robust baseline        |
| XGBoost       | Supervised   | $O(B \cdot m \cdot n)$              | $O(B \cdot \text{depth})$ | Partial (SHAP) | Kaggle competitions, tabular         |
| k-Means       | Unsupervised | $O(n \cdot k \cdot l \cdot m)$      | $O(k \cdot n)$            | Yes            | Spherical clusters, known            |

# Assessment Structure

## Final Examination

Comprehensive, 3 hours. Theory + algorithm application.

40%

## Coursework Project

End-to-end ML pipeline on a real dataset (teams of 2–3).

30%

## Lab Assignments (×4)

Python implementations: k-NN, CNN, k-Means, FP-Growth.

20%

## Midterm Test

Modules 1–3. MCQ + short answer.

10%

**Tools Required:** Python 3.10+ · scikit-learn · PyTorch or TensorFlow · pandas · NumPy · matplotlib · SHAP · Optuna · Jupyter Notebook

# Sample Examination Questions

Q1. Derive the Bayes optimal decision rule for a two-class problem. What does the Bayes error rate represent, and why is it rarely achievable in practice?

12 pts

Q2. Compare k-Means and DBSCAN clustering. For a dataset with two crescent-shaped clusters, which algorithm should be used and why?

10 pts

Q3. A fraud detection dataset has 0.1% positive cases. Why is accuracy inappropriate here? Name THREE techniques to handle the class imbalance and TWO better evaluation metrics.

15 pts

Q4. Explain the bias-variance trade-off. A decision tree with `max_depth=None` achieves 100% train accuracy but 62% test accuracy. Diagnose and prescribe a fix.

10 pts

Q5. Describe the FP-Growth algorithm. What is the key advantage over Apriori, and what property enables it?

8 pts

# Required Tools & Python Ecosystem

## Core ML

- scikit-learn — classical ML algorithms
- XGBoost / LightGBM — gradient boosting
- imbalanced-learn — SMOTE, resampling
- mlxtend — association rule mining

## Deep Learning

- PyTorch — flexible, research-first
- TensorFlow / Keras — production pipelines
- Hugging Face Transformers — BERT/GPT
- torchvision — CNN architectures

## Explainability & Tuning

- SHAP — Shapley value explanations
- LIME — local surrogate explanations
- Optuna / Hyperopt — Bayesian HPO
- Weights & Biases — experiment tracking

## Data & Visualisation

- pandas — data manipulation
- NumPy — numerical computing
- matplotlib / seaborn — static plots
- Plotly — interactive visualisations

# Emerging Trends in Pattern Recognition



## Foundation Models

GPT-4, CLIP, SAM — pre-trained on billions of examples; few-shot learning across tasks with minimal fine-tuning.



## AutoML

Automated feature engineering, model selection, and HPO.  
Tools: AutoSklearn, H2O AutoML, Google AutoML.



## Federated Learning

Train ML models across decentralised devices without sharing raw data — critical for healthcare and finance privacy.



## Graph Neural Networks

Learning on graphs: molecular property prediction, social network analysis, knowledge graph completion.



## Multimodal Learning

Models combining text + image + audio + video. CLIP (OpenAI) maps images and text to shared embedding space.



## Explainable AI (XAI)

Growing regulatory demand (GDPR, EU AI Act) for models that can explain their decisions transparently.



# Thank You

---

CSD622 · Pattern Recognition & Data Mining

*From Bayes to deep learning — data reveals its patterns to those who look carefully.*